LAB 10 HTTP REST APIs, JSON and offline fallback

Unit 4 • 3-hour guided laboratory • CLO-5

Learning goal. Retrieve remote JSON with robust error/loading states and preserve a last-successful response for offline reading.

Core concepts. http Client, HTTPS and Android INTERNET permission, JSON mapping, FutureBuilder, timeouts, HTTP status codes, shared_preferences cache.

Before you start. A phone with internet access; network API examples use public JSONPlaceholder https://jsonplaceholder.typicode.com and may fail under restrictive networks.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android mad_lab10
cd mad_lab10
flutter pub add http shared_preferences
```

Android network permission

For both tasks, add the following line DIRECTLY inside the <manifest ...> element and outside the <application> element of android/app/src/main/AndroidManifest.xml. Preserve other manifest content; do not replace the generated file wholesale.

android/app/src/main/AndroidManifest.xml — insert inside <manifest>

```
<uses-permission android:name="android.permission.INTERNET" />
```

Solved task 1 | Fetch and parse live posts using a typed model

Problem and objective. Call a GET endpoint, check its status, decode a JSON array into Post objects, and expose loading, success and failure states.

lib/main.dart

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;

class Post {
  final int id;
  final String title;
  const Post(this.id, this.title);
  factory Post.fromJson(Map<String, dynamic> json) =>
      Post(json['id'] as int, json['title'] as String);
}

Future<List<Post>> fetchPosts() async {
  final response = await http.get(
    Uri.parse('https://jsonplaceholder.typicode.com/posts'))
      .timeout(const Duration(seconds: 12));
  if (response.statusCode != 200) {
    throw Exception('Server returned ${response.statusCode}');
  }
  final values = jsonDecode(response.body) as List<dynamic>;
  return values.map((e) => Post.fromJson(e as Map<String, dynamic>))
      .toList();
}

void main() => runApp(const MaterialApp(home: PostsPage()));

class PostsPage extends StatefulWidget {
  const PostsPage({super.key});
  @override
  State<PostsPage> createState() => _PostsPageState();
}

class _PostsPageState extends State<PostsPage> {
  late Future<List<Post>> request;
  @override
  void initState() { super.initState(); request = fetchPosts(); }

  @override
  Widget build(BuildContext context) => Scaffold(
```

```
    appBar: AppBar(title: const Text('REST posts')),
    floatingActionButton: FloatingActionButton(
      onPressed: () => setState(() => request = fetchPosts()),
      child: const Icon(Icons.refresh)),
    body: FutureBuilder<List<Post>>(
      future: request,
      builder: (context, snapshot) {
        if (snapshot.connectionState != ConnectionState.done) {
          return const Center(child: CircularProgressIndicator());
        }
        if (snapshot.hasError) {
          return Center(child: Text('Request failed: ${snapshot.error}'));
        }
        final posts = snapshot.data ?? [];
        return ListView.builder(itemCount: posts.length,
          itemBuilder: (_, i) => ListTile(
            leading: CircleAvatar(child: Text('${posts[i].id}')),
            title: Text(posts[i].title),
          ));
      },
    ),
  );
}
```

How the solution works. HTTP returns bytes/text and a status code; jsonDecode transforms text into Dart structures, and the factory enforces a consistent typed model. A stored Future in State prevents repeated HTTP calls every time build executes. timeout avoids indefinite waiting and FutureBuilder describes the asynchronous lifecycle.

Expected output. A scrolling list of sample posts appears online; airplane mode plus Refresh produces a friendly error, not an uncaught crash.

Check your work. Record behavior with the network available and unavailable; identify the status code handling branch.

Solved task 2 | Offline-first bulletin cache

Problem and objective. Save the last successful bulletin response in preferences. When the API fails, show the cached copy with an explicit offline label.

lib/main.dart

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'package:shared_preferences/shared_preferences.dart';

void main() => runApp(const MaterialApp(home: OfflinePage()));

class OfflinePage extends StatefulWidget {
  const OfflinePage({super.key});
  @override
  State<OfflinePage> createState() => _OfflinePageState();
}

class _OfflinePageState extends State<OfflinePage> {
  final prefs = SharedPreferencesAsync();
  List<String> titles = [];
  String status = 'Loading...';
  bool busy = false;

  @override
  void initState() { super.initState(); refresh(); }

  Future<void> refresh() async {
    if (busy) return;
    setState(() => busy = true);
    try {
      final response = await http.get(
        Uri.parse('https://jsonplaceholder.typicode.com/posts'))
          .timeout(const Duration(seconds: 10));
      if (response.statusCode != 200) throw Exception('HTTP error');
      final decoded = jsonDecode(response.body) as List<dynamic>;
      final latest = decoded.take(15).map((e) =>
        (e as Map<String, dynamic>)['title'] as String).toList();
      await prefs.setStringList('cached_bulletins', latest);
      if (mounted) setState(() {
        titles = latest; status = 'Online • Updated from API';
      });
    } catch (_) {
```

```
      final cached = await prefs.getStringList('cached_bulletins') ?? [];
      if (mounted) setState(() {
        titles = cached;
        status = cached.isEmpty ? 'Offline • No cached data yet'
            : 'Offline • Showing saved copy';
      });
    } finally {
      if (mounted) setState(() => busy = false);
    }
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Offline bulletins')),
    body: Column(children: [
      ListTile(title: Text(status),
        trailing: IconButton(onPressed: busy ? null : refresh,
          icon: const Icon(Icons.refresh))),
      if (busy) const LinearProgressIndicator(),
      Expanded(child: ListView.builder(
        itemCount: titles.length,
        itemBuilder: (_, i) => ListTile(title: Text(titles[i])),
      )),
    ]),
  );
}
```

How the solution works. The network is authoritative when reachable. After a validated successful response, the app caches 15 titles. The exception path reads the last saved values. Users see a data-freshness indicator so old information is not mistaken for fresh data. Persistent cache is a fallback, not a full synchronization engine.

Expected output. Open online once; then enable airplane mode, press Refresh and see saved titles labeled Offline. On a clean offline install, see No cached data yet.

Check your work. Test online-first, offline-after-cache and first-run-without-internet separately.

API correctness and security review

Treat API responses as untrusted: check status and type, handle timeout and decoding errors, use HTTPS, do not embed secret API tokens in the Flutter client, and distinguish connection failure from unauthorized response. If the remote contract changes, change the model parser and add tests before shipping.

Scenario-based practice — unsolved

Scenario 1. Bus timetable cache. The timetable must work without internet and warn users if data is older than one day. Specify a timestamp schema, refresh policy and conflict behavior.

Scenario 2. Account-specific API. Consume a bearer-token endpoint without placing long-lived secrets in app source. Identify storage, token refresh, HTTP 401 handling and logout behavior.

Technical viva — justify your reasoning

1. Why is an HTTP 200 response not a guarantee of valid JSON shape?
2. Why should a Future not be created directly on every build?
3. What distinguishes offline-first from online-only with a spinner?
4. Why can caching stale medical or emergency data be dangerous?
5. What is the security difference between an API key baked into an app and a server-held key?
6. Where should Android INTERNET permission be declared?

Submission and self-check

Demonstrate both online fetch and cached offline fallback, with Android permission and a short error-state test log.

CSC303 / NOVEMBER 2026 REVISION

LAB 10 EXTENSION | Correct cache semantics and error tests

The solved network examples implement **network-first with fallback**. Offline-first means the UI is driven by the local store and network synchronization occurs separately. Do not label a timeout-then-cache flow as a complete offline-first design.

Safe ordering for a last-known-good cache

```
Future<List<Post>> fetchValidatedPosts(http.Client client) async {
  final response = await client.get(
    Uri.parse('https://jsonplaceholder.typicode.com/posts'),
  ).timeout(const Duration(seconds: 10));
  if (response.statusCode != 200) {
    throw Exception('HTTP ${response.statusCode}');
  }
  final decoded = jsonDecode(response.body) as List<dynamic>;
  final posts = decoded.take(25).map((row) =>
      Post.fromJson(row as Map<String, dynamic>)).toList();
  // Now, and only now, save response.body as the new cache.
  return posts;
}
```

This is a pattern: adapt the model name `Post` to the actual model from the task. Validate the required fields and types, not just the HTTP code. Keep a usable cache when parsing fails, and preserve a status indicator explaining whether data is fresh or cached.

Minimum network and persistence test matrix

Case	Expected result
Valid HTTP 200 with expected JSON	Display typed records and update cache
HTTP 500 / timeout / disconnected	Display previous valid cache, visibly marked stale
HTTP 200 with malformed JSON	Do NOT replace previous cache; show fallback/error
No network and no cache	Render an informative empty/error state; retry is available
App restart while offline	Persisted cache remains accessible
Unexpected fields / missing required field	Handle as an invalid record/response without an uncaught crash

Implementation boundary

For the final project, put the HTTP client behind an injectable repository and persist typed records in Drift/SQLite or Firestore as required by the project brief. SharedPreferencesAsync is acceptable for a demonstration cache but does not replace the required relational/cloud storage layer in the final project.